

ATIVIDADE DE REVISÃO

Contextualização: Sistema TicketCode

A empresa **DevSolutions** é uma software house de médio porte que atende clientes de vários segmentos. Hoje, quando um cliente reporta um problema, uma melhoria ou uma dúvida, o atendente anota em um bloco de papel e repassa por e-mail para o time de suporte. Não há controle de prioridade, prazo de atendimento nem histórico organizado.

O diretor de operações decidiu que é hora de modernizar e contratou você para desenvolver o **TicketCode**, o sistema interno de suporte de TI da empresa. O sistema deve ser capaz de abrir tickets com diferentes configurações e prioridades, calcular automaticamente o prazo de atendimento de acordo com o nível de urgência, notificar o solicitante e a fila de atendimento quando um ticket for aberto, se integrar ao sistema de e-mail já utilizado pela empresa que tem uma interface diferente do padrão do TicketCode e aplicar comportamentos de log e validação sem modificar o serviço principal.

O projeto é **backend Java puro** sem Spring Boot, sem banco de dados, sem framework externo. Apenas Java e orientação a objetos.

Estrutura esperada do projeto

ticketcode/

└─ src/

└─ main/

└─ java/

└─ br/com/ticketcode/

└─ model/

└─ factory/

└─ builder/

└─ adapter/

└─ decorator/

└─ strategy/

└─ observer/

└─ Main.java

ENTREGÁVEL 1 — Builder: Abertura de Ticket

Abrir um ticket exige alguns dados obrigatórios para que o time de suporte consiga atuar. Outros dados são opcionais e enriquecem o ticket mas não impedem sua criação. O sistema não pode aceitar um ticket sem as informações mínimas necessárias.

O que precisa existir

Classe Ticket com os seguintes atributos:

Atributo	Tipo	Obrigatoriedade
numeroTicket	int	Obrigatório

Atributo	Tipo	Obrigatoriedade
titulo	String	Obrigatório
descricao	String	Obrigatório
emailSolicitante	String	Obrigatório
departamento	String	Opcional — padrão: "Geral"
versaoSistema	String	Opcional — padrão: "Não informada"
anexos	List<String>	Opcional — padrão: lista vazia
urgente	boolean	Opcional — padrão: false

Regras do build():

- Lançar Exception se titulo for nulo ou vazio
- Lançar Exception se emailSolicitante for nulo ou não contiver @
- Lançar Exception se descricao for nula ou vazia
- Lançar Exception se numeroTicket for menor ou igual a zero

Requisitos adicionais:

- Ticket deve ser **imutável** após o build() — sem setters, lista de anexos retornada como cópia não modificável
- O construtor de Ticket deve ser **privado**
- Implementar toString() exibindo todos os atributos de forma legível

Demonstração obrigatória no Main.java: criar 2 tickets, um com todos os atributos preenchidos e outro apenas com os obrigatórios. Demonstrar também que uma tentativa de criar um ticket com e-mail inválido lança a exceção esperada usando try/catch.

ENTREGÁVEL 2 — Factory Method: Tipos de Ticket

A DevSolutions categoriza os tickets em tipos distintos. Cada tipo tem um nome, uma descrição do que representa e um peso interno que influencia a fila de atendimento. O sistema deve criar o tipo correto de ticket a partir da categoria informada pelo atendente, sem que nenhuma outra parte do código precise conhecer as classes concretas de cada tipo.

O que precisa existir

Interface CategoriaTicket com os métodos:

- String getNome()
- String getDescricao()
- int getPeso() — número inteiro que representa a "gravidade base" da categoria

Enum TipoCategoria com os valores: BUG, MELHORIA, DUVIDA, ACESSO, INFRAESTRUTURA

Implementações concretas — uma para cada valor do enum. Cada uma deve retornar nome, descrição e peso coerentes com o que a categoria representa. BUG, por exemplo, tem peso maior que DUVIDA.

Classe CategoriaTicketFactory com o método estático `criar(TipoCategoria tipo)` que retorna a implementação correta. Deve lançar `Exception` para tipo nulo.

Requisito adicional: nenhuma classe fora da `Factory` deve instanciar diretamente nenhuma implementação concreta de `CategoriaTicket`.

Demonstração obrigatória no Main.java: criar ao menos 3 categorias diferentes via `factory` e exibir nome, descrição e peso de cada uma.

ENTREGÁVEL 3 — Strategy: Cálculo de Prazo de Atendimento (SLA)

Todo ticket aberto precisa ter um prazo máximo de atendimento calculado automaticamente com base no nível de urgência definido pelo atendente. Esse prazo é chamado de SLA (Service Level Agreement). As regras variam por nível e o sistema precisa suportar novos níveis sem que a classe que calcula o prazo precise ser modificada.

Níveis e regras

Nível	Regra de prazo
CRITICO	Prazo de 4 horas a partir da abertura
ALTO	Prazo de 8 horas a partir da abertura
MEDIO	Prazo de 24 horas a partir da abertura
BAIXO	Prazo de 72 horas a partir da abertura
PLANEJADO	Prazo de 168 horas (7 dias) a partir da abertura

O que precisa existir

Interface CalculadoraDeSLA com os métodos:

- `LocalDateTime calcularPrazo(LocalDateTime abertura)` — recebe quando o ticket foi aberto e retorna o prazo limite
- `String getNivelDescricao()` — retorna uma descrição legível do nível, ex: "CRÍTICO, Atendimento em até 4 horas"

Enum NivelUrgencia com os 5 valores acima.

Implementação concreta para cada nível, cada uma soma o número de horas corretas ao `LocalDateTime` recebido.

Classe SLAStrategyFactory com o método estático `criar(NivelUrgencia nivel)`.

Classe GerenciadorDeSLA com o método `void exibirPrazo(Ticket ticket, CalculadoraDeSLA estrategia)` que imprime no console o número do ticket, o nível aplicado e o prazo calculado de forma legível.

Requisito adicional: `GerenciadorDeSLA` não deve conter nenhum `if` ou `switch` para calcular ou escolher o prazo — toda a lógica está na estratégia recebida.

Demonstração obrigatória no Main.java: calcular e exibir o prazo para o mesmo ticket usando 3 níveis diferentes.

ENTREGÁVEL 4 — Adapter: Integração com o Sistema de E-mail

A `DevSolutions` já usa um sistema de e-mail de um fornecedor externo para comunicação interna. Esse sistema existe e funciona mas foi desenvolvido por terceiros, seu código não pode ser modificado. O

TicketCode precisa enviar notificações por e-mail usando a interface interna do sistema, e um componente deve fazer a ponte entre os dois contratos.

Interface interna do TicketCode (o que o sistema conhece)

Interface NotificadorDeTicket com os métodos:

- void notificarAbertura(Ticket ticket, String prazoFormatado)
- void notificarEncerramento(int numeroTicket, String resolucao)

Classe do fornecedor externo — não pode ser modificada

// Simula o sistema de e-mail do fornecedor — não altere esta classe

```
public class SistemaEmailFornecedor {  
  
    public void enviarMensagem(String destinatario, String assunto, String corpo) {  
        System.out.println("[EMAIL FORNECEDOR] Para: " + destinatario);  
        System.out.println("[EMAIL FORNECEDOR] Assunto: " + assunto);  
        System.out.println("[EMAIL FORNECEDOR] Corpo: " + corpo);  
    }  
  
    public void enviarAlerta(String destinatario, String codigoAlerta, String detalhe) {  
        System.out.println("[EMAIL FORNECEDOR] ALERTA " + codigoAlerta  
            + " para " + destinatario + ": " + detalhe);  
    }  
}
```

O que precisa existir

A classe SistemaEmailFornecedor acima deve existir exatamente como está no projeto — não é modificada.

Classe EmailNotificadorAdapter que:

- Implementa NotificadorDeTicket
- Internamente usa uma instância de SistemaEmailFornecedor
- No notificarAbertura: usa enviarMensagem com o e-mail do solicitante como destinatário, "Ticket #X aberto" como assunto, e uma mensagem com o título e o prazo como corpo
- No notificarEncerramento: usa enviarAlerta com o e-mail do solicitante como destinatário, "TICKET_FECHADO" como código de alerta, e o número do ticket + resolução como detalhe

Requisito adicional: nenhuma classe do sistema exceto o Adapter deve ter acesso direto a SistemaEmailFornecedor.

Demonstração obrigatória no Main.java: notificar a abertura de um ticket e depois seu encerramento, usando apenas a interface NotificadorDeTicket.

ENTREGÁVEL 5 — Decorator: Serviço de Tickets

O serviço que registra um ticket no sistema precisa de dois comportamentos adicionais que não fazem parte da regra de negócio central: um log de auditoria registrando cada operação com data e hora, e uma validação que impede o registro de tickets duplicados com o mesmo número. Esses comportamentos devem ser adicionados de forma independente, sem modificar o serviço original.

O que precisa existir

Interface ServicoDeTicket com o método:

- `String registrar(Ticket ticket)` — retorna uma mensagem de confirmação, ex: "Ticket #5 registrado com sucesso"

Classe TicketServiceImpl que implementa `ServicoDeTicket`:

- É a implementação real — registra o ticket internamente numa `List<Ticket>` e retorna a confirmação
- Não tem nenhuma responsabilidade de log ou validação de duplicidade

Classe LogTicketDecorator que implementa `ServicoDeTicket`:

- Recebe um `ServicoDeTicket` no construtor
- Antes de delegar, imprime: "[LOG] {dataHora} — Registrando ticket #{numero} — Solicitante: {email}"
- Após a delegação, imprime: "[LOG] {dataHora} — Ticket #{numero} processado"
- Retorna o resultado do serviço interno

Classe ValidacaoDuplicidadeDecorator que implementa `ServicoDeTicket`:

- Recebe um `ServicoDeTicket` no construtor
- Mantém internamente um `Set<Integer>` com os números de tickets já registrados
- Se o número do ticket já existir no Set, **não delega** e lança `IllegalStateException` com a mensagem "Ticket #{numero} já existe no sistema"
- Se o número for novo, adiciona ao Set, delega e retorna o resultado

Demonstração obrigatória no Main.java: compor os decorators (Log envolve Validação que envolve o `TicketServiceImpl`), registrar dois tickets válidos e tentar registrar um ticket com número repetido — demonstrando que a exceção é lançada pelo Decorator e o log ainda é impresso antes dela.

ENTREGÁVEL 6 — Observer: Evento de Ticket Aberto

Valor: 2,0 pontos

Contexto

Quando um ticket é oficialmente aberto no sistema, três partes independentes precisam ser notificadas automaticamente: o solicitante recebe confirmação por e-mail, a fila de atendimento registra o novo ticket com sua urgência, e o painel de métricas contabiliza os tickets abertos por categoria. O serviço que abre o ticket não deve conhecer nenhum desses três comportamentos.

O que precisa existir

Classe TicketAbertoEvento — imutável — com os atributos:

- `numeroTicket` (int)
- `emailSolicitante` (String)
- `tituloTicket` (String)
- `nomeCategoria` (String)
- `nivelUrgencia` (`NivelUrgencia` — o enum do Entregável 3)
- `prazoAtendimento` (`LocalDateTime`)
- `aberturaEm` (`LocalDateTime` — inicializado automaticamente com `LocalDateTime.now()`)

Interface TicketAbertoObserver com o método:

- void aoAbrirTicket(TicketAbertoEvento evento)

Três implementações concretas:

ConfirmacaoSolicitanteObserver:

- Imprime: "[CONFIRMAÇÃO] E-mail enviado para {email} — Ticket #{numero} recebido. Prazo: {prazo}"

FilaAtendimentoObserver:

- Mantém internamente uma List<String> representando a fila
- Ao receber o evento, adiciona o ticket na fila com o formato: "#X | {titulo} | {nivel} | Prazo: {prazo}"
- Imprime: "[FILA] Ticket #{numero} adicionado. Total na fila: X ticket(s)"
- Tem método público exibirFila() que imprime todos os tickets da fila

MetricasObserver:

- Mantém internamente um Map<String, Integer> que conta tickets abertos por categoria
- A cada evento incrementa o contador da categoria correspondente
- Imprime: "[MÉTRICAS] Categoria '{categoria}': X ticket(s) aberto(s) no total"

Classe TicketEventPublisher que:

- Mantém uma List<TicketAbertoObserver>
- Tem método registrar(TicketAbertoObserver observer)
- Tem método publicarTicketAberto(TicketAbertoEvento evento) que notifica todos os observers
- **Garante** que exceção em um observer não interrompe a notificação dos demais

Classe TicketFacade — ponto central do sistema — com o método:

- void abrirTicket(Ticket ticket, NivelUrgencia nivel, CategoriaTicket categoria): calcula o SLA usando SLAStrategyFactory, cria o evento com todas as informações e publica via publisher
- Recebe TicketEventPublisher e CalculadoraDeSLA no construtor — ou recebe apenas o publisher e cria internamente a Strategy com base no NivelUrgencia

Demonstração obrigatória no Main.java:

- Registrar os 3 observers no publisher
- Abrir ao menos 4 tickets com categorias e urgências diferentes
- Após as aberturas, chamar exibirFila() do FilaAtendimentoObserver para mostrar todos os tickets na fila

ENTREGÁVEL 7 — Main.java: Demonstração Integrada

O Main.java deve executar um fluxo completo demonstrando **todos os 6 padrões integrados** na seguinte sequência mínima:

1. Criar 2 categorias via CategoriaTicketFactory (Factory Method)
2. Montar 2 tickets via Ticket.Builder — um urgente com anexos, outro simples (Builder)
3. Calcular e exibir o SLA de cada ticket com níveis diferentes (Strategy)
4. Notificar abertura dos tickets via EmailNotificadorAdapter (Adapter)

5. Registrar os tickets via serviço com decorators de log e validação (Decorator)
6. Abrir os tickets via TicketFacade e observar as reações dos 3 observers (Observer)

O programa deve rodar **sem exceção não tratada** e a saída no console deve identificar claramente cada padrão em funcionamento.